

DW_sqrt_seq

Sequential Square Root

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

Features and Benefits

- Parameterized word length
- Parameterized number of clock cycles
- Unsigned and signed (two's complement) square roots
- Registered or un-registered inputs and outputs
- Includes a low-power implementation (at a sub-level) that has power benefits from minPower optimization (for details, see [Table 1-3](#) on page 2)

Revision History

Description

DW_sqrt_seq is a sequential square root designed for low area, area-time trade-off, or high frequency (small cycle time) applications. Note that data input is taken as absolute value. Two's complement input is converted into unsigned magnitude. Output is unsigned (positive).

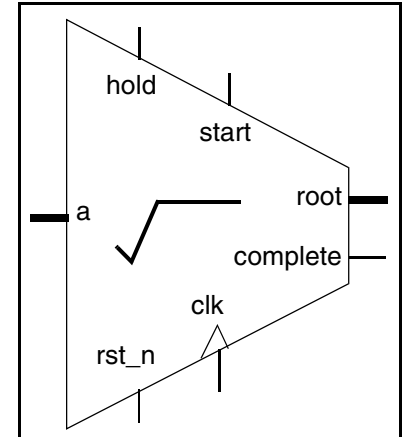


Table 1-1 Pin Description

Pin Name	Width	Direction	Function
clk	1 bit	Input	Clock
rst_n	1 bit	Input	Reset, active low
hold	1 bit	Input	Hold current operation (=1)
start	1 bit	Input	Start operation (=1) A new operation is started by setting start = 1 for one clock cycle.
a	<i>width</i> bits	Input	Radicand
complete	1 bit	Output	Operation completed (=1)
root	$(width + 1)/2$ bits	Output	Square root

Table 1-2 Parameter Description

Parameter	Values	Description
width	≥ 6	Word length of a
tc_mode	0 or 1 Default: 0	Two's complement control 0: Unsigned 1: Two's complement
num_cyc	3 to $\text{int}((\text{width} + 1)/2)$ Default: 3	User-defined number of clock cycles to produce a valid result The real number of clock cycles depends on various parameters and is given in Table 1-6 on page 4 and the topic titled “ Formula for Radicand Bits Processed Per Cycle ” on page 4.
rst_mode	0 or 1 Default: 0	Reset mode 0: Asynchronous reset 1: Synchronous reset
input_mode ^a	0 or 1 Default: 1	Registered inputs 0: No 1: Yes
output_mode	0 or 1 Default: 1	Registered outputs 0: No 1: Yes
early_start	0 or 1 Default: 0	Computation start 0: Start computation in the second cycle 1: Start computation in the first cycle For the dependency of <i>early_start</i> on <i>input_mode</i> , see Table 1-6 on page 4.

- a. When configured with the parameter *input_mode* set to '0', input 'a' MUST be held constant from the time *start* is asserted until *complete* has gone high to signal completion of the calculation. Conversely, if a configuration with the parameter *input_mode* set to '1' is used, the 'a' input is captured when *start* is high and otherwise ignored.

Table 1-3 Synthesis Implementations

Implementation	Function	License Feature Required
cpa ^a	Carry-propagate adder synthesis model	DesignWare ^b

- a. To achieve low-power benefits in sub-module implementations, you need to enable *minPower*; for details, see “[Enabling minPower](#)” on page 9.
- b. For releases prior to P-2019.03, the DesignWare-LP license feature is required to achieve low-power benefits.

Table 1-4 Simulation Models

Model ^a	Function
DW03.DW_SQRT_SEQ_CFG_SIM	Design unit name for VHDL simulation
dw/dw03/src/DW_sqrt_seq_sim.vhd	VHDL simulation model source code
dw/sim_ver/DW_sqrt_seq.v	Verilog simulation model source code

- a. Note that during the computation phase (after start and before complete is asserted), the simulation models output X values and therefore cannot be used as a compare for gate-level simulations.

Table 1-5 Operation Truth Table

start	hold	Operation
0	0	Idle or Running
0	1	Hold
1	X	Start

DW_sqrt_seq computes the integer square root of radicand *a* in a user-defined number of clock cycles (*num_cyc*). As long as *start*=1 the square root operation is in the initialization state. Once *start* = 0, the calculation begins followed by valid output flagged when *complete* = 1 or an intervening setting of *start* = 1. The square root operation is stalled when *hold* = 1. For theory of square root operation, refer to the datasheet for DW_sqrt.

The parameter *tc_mode* determines whether the data of input (*a*) is interpreted as unsigned (*tc_mode*=0) or two's complement (*tc_mode*=1) number. The input is converted into unsigned absolute value for calculation of square root.

The internal registers can either have an asynchronous (*rst_mode* = 0) or synchronous reset (*rst_mode* = 1) that is connected to the reset signal *rst_n*.

After reset conditions are released (*rst_n* = 1) there are not restrictions on when *start* can be set to 1 and then to 0. However, if *start* is set to 0 immediately after *rst_n* goes to 1 and *start*=0 continues through the first *num_cyc* clock cycles, then *complete* will go to 1. This first *complete*=1 when no *start* is initiated following reset may yield invalid results and should be disregarded.

The parameter *input_mode* determines whether the inputs are to be registered inside DW_sqrt_seq (*input_mode* = 1) or not (*input_mode* = 0). If configured without input registers (*input_mode* = 0), then the logic that drives input *a* must hold the input value constant for the entire time it takes to calculate the result (from the cycle before *start* drops until *complete* goes high). When configured with input registers (*input_mode* = 1) inputs *a* is captured when *start* is high and ignored until *start* goes high again.

**Note**

When configured with no input registers, changes on input *a* while *complete* is low (calculation cycle) will produce unpredictable output values. Simulation models will produce unknown output values (Xs) and post an error message indicating the instance that violated this rule and the simulation time when the violation was detected.

The parameter *output_mode* determines whether the outputs are registered (*output_mode* = 1) or not (*output_mode* = 0). When the parameter *early_start* = 1, computation starts immediately after setting *start* to 1. This saves one extra cycle to store the data (*early_start* = 0), but feeds the inputs directly into the components critical path. Table 1-6 on page 4 shows the *input_mode*, *output_mode*, and *early_start* parameter combinations and corresponding actual number of cycles required to perform an operation.

Table 1-6 Actual Cycles Based on *input_mode*, *output_mode*, and *early_start*

<i>input_mode</i>	<i>output_mode</i>	<i>early_start</i>	Actual Number of Cycles
0	0	0	<i>num_cyc</i> -2
0	0	1	Invalid parameter setting
0	1	0	<i>num_cyc</i> -1
0	1	1	Invalid parameter setting
1	0	0	<i>num_cyc</i> -1
1	0	1	<i>num_cyc</i> -2
1	1	0	<i>num_cyc</i>
1	1	1	<i>num_cyc</i> -1

Note that the *num_cyc* value indicates the actual throughput of the device from when *start* is asserted to when *complete* is asserted. However, if a calculation is in progress (before the *num_cyc* number of cycles has been reached) when *start* is asserted again, the results are undetermined until *complete* is asserted. The results associated with the assertion of *complete* are from the input values from the previous assertion of *start*.

Formula for Radicand Bits Processed Per Cycle

The following formula describes the number of radicand bits processed per cycle:

$$\text{bits processed per cycle} = \text{ceil} \left(\text{width} / \text{num_cyc} \right)$$

where:

width is the bit width of the radicand (as defined in Table 1-2 on page 2)

num_cyc is the number of clock cycles required for square root (as defined in Table 1-2 on page 2)

Note that there is a restriction of the relationship between *width* and *num_cyc* such that $\text{num_cyc} \leq \text{int}((\text{width} + 1) / 2)$. In other words, *num_cyc* cannot exceed 1/2 of *width*.

Formula for Actual Number of Cycles Required

The actual number of clock cycles required for a computation is calculated using the following formula:

$$\text{actual number of cycles required} = \text{num_cyc} - (1 - \text{output_mode}) - (1 - \text{input_mode}) - \text{early_start}$$

where:

num_cyc is the number of clock cycles required for square root (as defined in [Table 1-2](#) on page 2)

output_mode is the control for registered output (as defined in [Table 1-2](#) on page 2)

input_mode is the control for registered inputs (as defined in [Table 1-2](#) on page 2)

early_start is the control for when the computation starts (as defined in [Table 1-2](#) on page 2)

Suppressing Warning Messages During Verilog Simulation

The Verilog simulation model includes macros that allow you to suppress warning messages during simulation.

To suppress all warning messages for all DWBB components, define the DW_SUPPRESS_WARN macro in either of the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_SUPPRESS_WARN
```

- Or, include a command line option to the simulator, such as:

+define+DW_SUPPRESS_WARN (which is used for the Synopsys VCS simulator)

The warning messages for this model include the following:

- If values other than 1 or 0 are present on a clock port, the following message is displayed:

```
WARNING: <instance_path>.<clock_name>_monitor:
        at time = <timestamp>, Detected unknown value, x, on <clock_name> input.
```

To suppress only this warning message for all DWBB components, use the following macro:

- Define the DW_DISABLE_CLK_MONITOR macro. You can define this macro in the following ways:

- Specify the Verilog preprocessing macro in Verilog code:

```
`define DW_DISABLE_CLK_MONITOR
```

- Or, include a command line option to the simulator, such as:

+define+DW_DISABLE_CLK_MONITOR (which is used for the Synopsys VCS simulator)

This message is also suppressed using the DW_SUPPRESS_WARN macro explained earlier.

- If the component is configured without an input register and an input operand changes during calculation, the following message is displayed:

WARNING: `<instance_path>`:
at time = `<timestamp>`, Operand input change on DW_sqrt_seq during calculation (configured without an input register) will cause corrupted results if operation is allowed to complete.

To suppress this message, use the DW_SUPPRESS_WARN macro explained earlier.

- If the component is configured without input and output registers and an input operand changes during calculation, the following message is displayed:

WARNING: `<instance_path>`:
at time = `<timestamp>`, Operand input change on DW_sqrt_seq during calculation (configured with neither input nor output register) causes output to no longer retain result of previous operation.

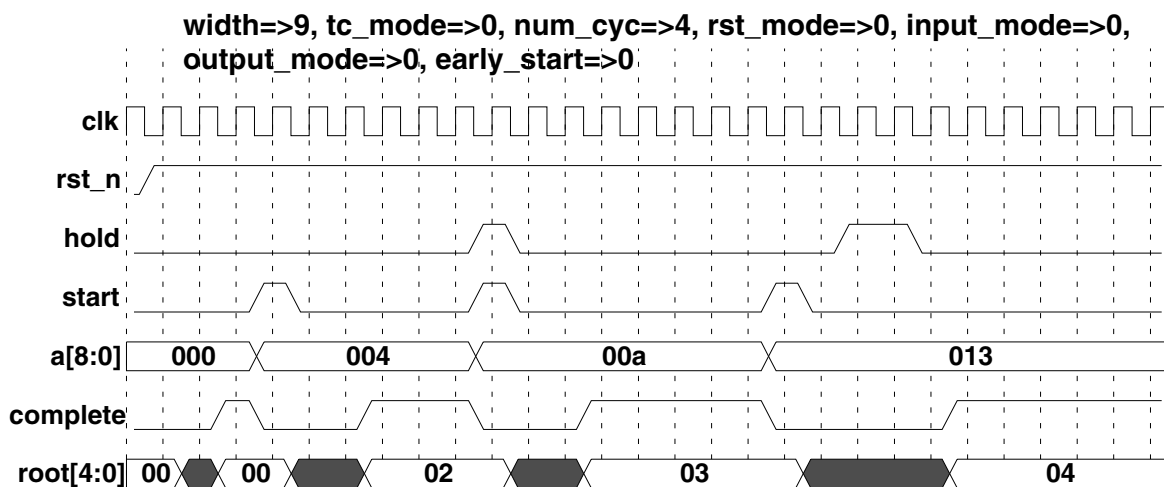
To suppress this message, use the DW_SUPPRESS_WARN macro explained earlier.

Timing Waveforms

The following timing waveforms show a 9-bit unsigned sequential square root for specific inputs of `hold` and `start` and their corresponding outputs. The parameter settings for each simulation are shown at the top of each figure. When `hold = 1` and `start = 0`, the result is delayed by the same number of clock cycles for which `hold` is 1. For example, if `hold = 1` for two clock cycles, then the result is delayed by two clock cycles.

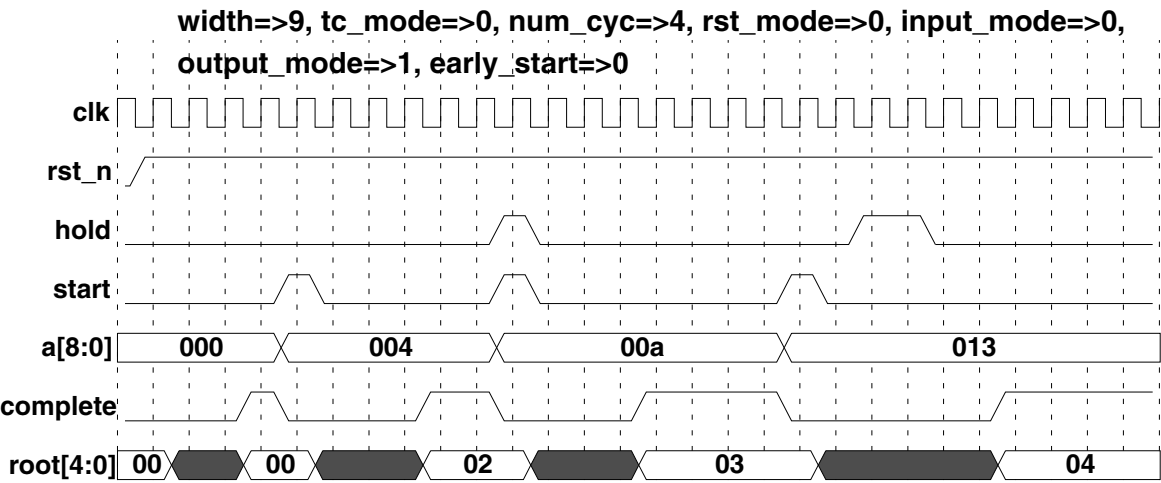
For the parameter settings shown in [Figure 1-1](#), [Table 1-6](#) on page 4 specifies that the result is produced after two cycles. However, data is available on the rising edge of the fourth clock following the clock that asserts the `start` signal.

Figure 1-1 Simulation Waveform 1



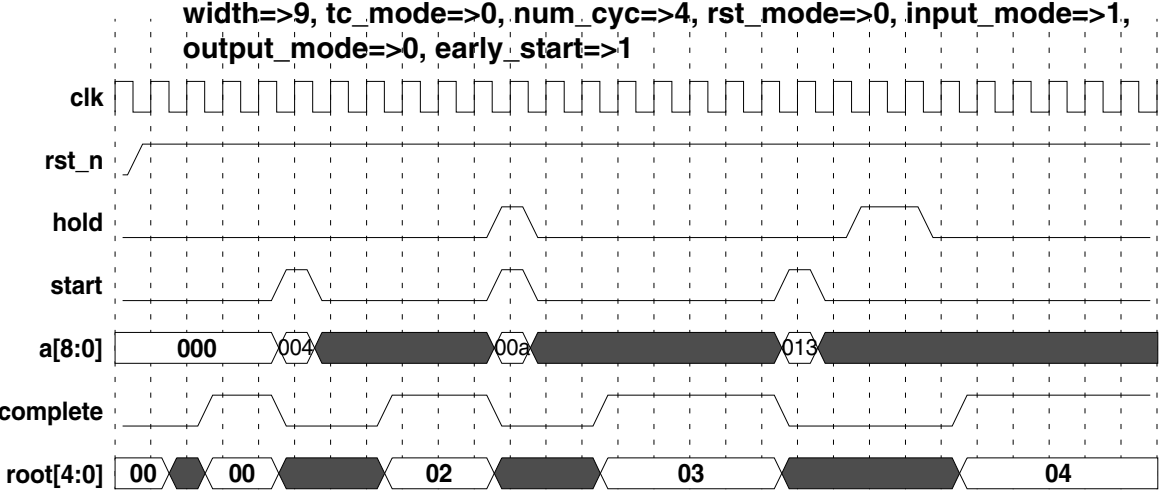
For the parameter settings shown in [Figure 1-2](#), [Table 1-6](#) on page 4 specifies that the result is produced after three cycles.

Figure 1-2 Simulation Waveform 2



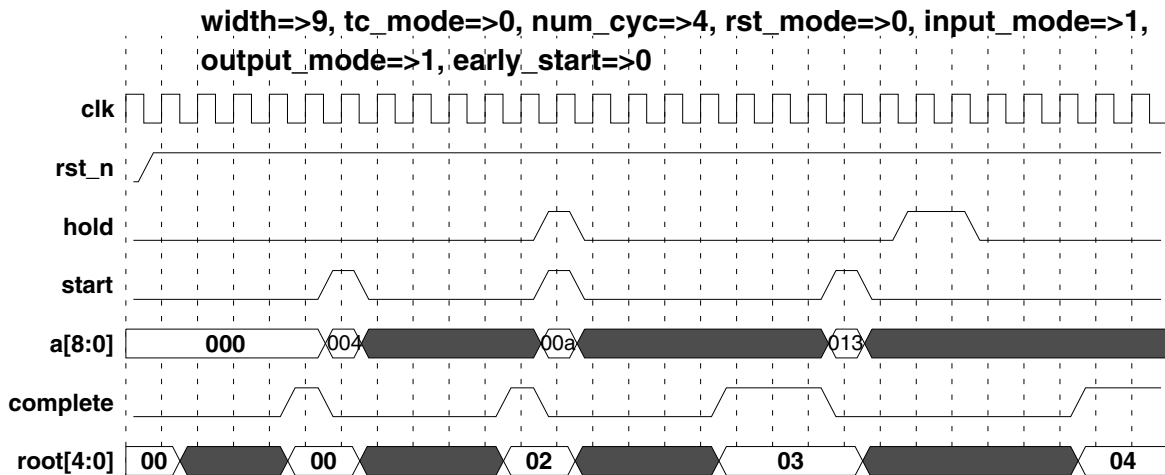
For the parameter settings shown in [Figure 1-3](#), [Table 1-6](#) on page 4 specifies that the result is produced after two cycles. Since *input_mode* = 1 (registered input) the input data can be removed after the first cycle.

Figure 1-3 Simulation Waveform 3



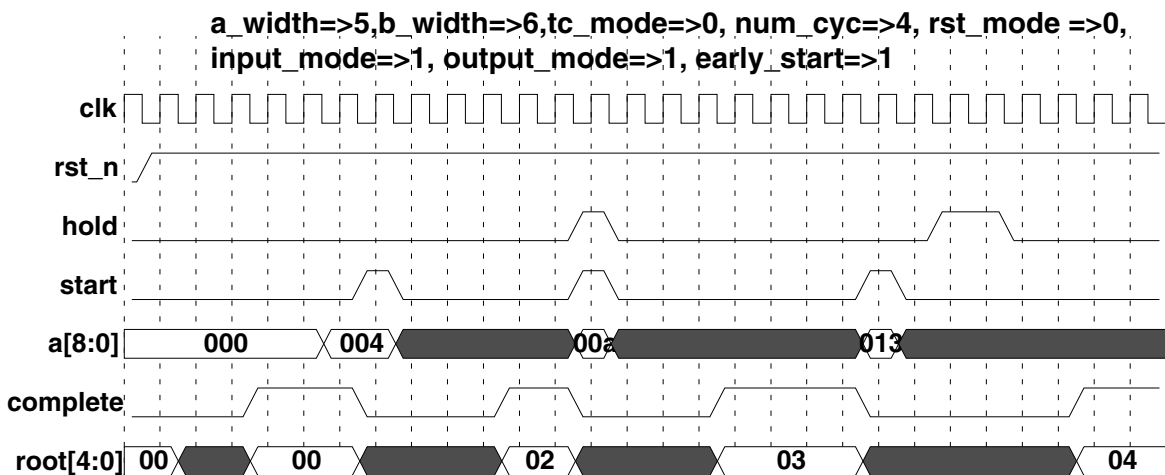
For the parameter settings shown in [Figure 1-4](#), [Table 1-6](#) on page 4 specifies the result is produced after four cycles. Since *input_mode* = 1 (registered input) the input data can be removed after the first cycle.

Figure 1-4 Simulation Waveform 4



For the parameter settings shown in [Figure 1-5](#) on page 8, [Table 1-6](#) on page 4 specifies that the result is produced after three clock cycles. Since *input_mode* = 1 (registered input), the input data can be removed after the first cycle. With *hold* = 1 and *start* = 1, the result is delayed by the same number of cycles that *hold* = 1. Note that the data will be available on the *num_cyc* number of clocks after the data is registered. Note that the data is registered in the clock cycle immediately preceding *start* = 1.

Figure 1-5 Simulation Waveform 5



Enabling minPower

You can instantiate this component without enabling minPower, but to achieve power savings from the low-power implementation (at a sub-level--see [Table 1-3](#) on page 2), you must enable minPower optimization, as follows:

- Design Compiler

- Version P-2019.03 and later:

- ```
set power_enable_minpower true
```

- Before version P-2019.03 (requires the DesignWare-LP license feature):

- ```
set synthetic_library {dw_foundation.sldb dw_minpower.sldb}  
set link_library {* $target_library $synthetic_library}
```

- Fusion Compiler

Optimization for minPower is enabled as part of the total_power metric setting. To enable the total_power metric, use the following:

```
set_qor_strategy -stage synthesis -metric total_power
```

Related Topics

- [Math – Sequential Overview](#)
- [DesignWare Building Block IP User Guide](#)

HDL Usage Through Component Instantiation - VHDL

```
library IEEE, DWARE;
use IEEE.std_logic_1164.all;
use DWARE.DW_Foundation_comp_arith.all;

entity DW_sqrt_seq_inst is
  generic (inst_width      : POSITIVE := 8; inst_tc_mode      : INTEGER := 0;
           inst_num_cyc    : INTEGER := 3; inst_rst_mode      : INTEGER := 0;
           inst_input_mode : INTEGER := 1; inst_output_mode    : INTEGER := 1;
           inst_early_start : INTEGER := 0 );
  port (inst_clk      : in std_logic; inst_rst_n : in std_logic;
        inst_hold     : in std_logic; inst_start : in std_logic;
        inst_a        : in std_logic_vector(inst_width-1 downto 0);
        complete_inst : out std_logic;
        root_inst     : out std_logic_vector((inst_width+1)/2-1 downto 0) );
end DW_sqrt_seq_inst;

architecture inst of DW_sqrt_seq_inst is
begin
  -- Instance of DW_sqrt_seq
  U1 : DW_sqrt_seq
    generic map (width => inst_width, tc_mode => inst_tc_mode,
                rst_mode => inst_rst_mode, input_mode => inst_input_mode,
                output_mode => inst_output_mode,
                early_start => inst_early_start )
    port map (clk => inst_clk, rst_n => inst_rst_n,
             hold => inst_hold, start => inst_start, a => inst_a,
             complete => complete_inst, root => root_inst );
end inst;

-- pragma translate_off
configuration DW_sqrt_seq_inst_cfg_inst of DW_sqrt_seq_inst is
  for inst
    end for;
end DW_sqrt_seq_inst_cfg_inst;
-- pragma translate_on
```

HDL Usage Through Component Instantiation - Verilog

```
module DW_sqrt_seq_inst(inst_clk, inst_rst_n, inst_hold, inst_start, inst_a,
                        complete_inst, root_inst );

    parameter inst_width = 8;
    parameter inst_tc_mode = 0;
    parameter inst_num_cyc = 3;
    parameter inst_rst_mode = 0;
    parameter inst_input_mode = 1;
    parameter inst_output_mode = 1;
    parameter inst_early_start = 0;

    // Please add +incdir+$SYNOPSIS/dw/sim_ver+ to your verilog simulator
    // command line (for simulation).

    input inst_clk;
    input inst_rst_n;
    input inst_hold;
    input inst_start;
    input [inst_width-1 : 0] inst_a;
    output complete_inst;
    output [(inst_width+1)/2-1 : 0] root_inst;

    // Instance of DW_sqrt_seq
    DW_sqrt_seq #(inst_width, inst_tc_mode, inst_num_cyc, inst_rst_mode,
                  inst_input_mode, inst_output_mode, inst_early_start)
    U1 (.clk(inst_clk), .rst_n(inst_rst_n), .hold(inst_hold),
        .start(inst_start), .a(inst_a), .complete(complete_inst),
        .root(root_inst) );
endmodule
```

Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet with visible change bars, click [here](#).

Date	Release	Updates
September 2024	DWBB_202409.0	Updated date and version; no technical changes
December 2023	DWBB_202312.0	Updated date and version; no technical changes
January 2023	DWBB_202212.1	- Clarified the note in Table 1-6 on page 4 - Clarified formulas in “Formula for Radicand Bits Processed Per Cycle” on page 4 “Formula for Actual Number of Cycles Required” on page 5
July 2020	DWBB_201912.5	Adjusted content and title of “Suppressing Warning Messages During Verilog Simulation” on page 5 and added the DW_SUPPRESS_WARN macro
October 2019	DWBB_201903.5	Added the “Disabling Clock Monitor Messages” section
March 2019	DWBB_201903.0	- Clarified license requirements in Table 1-3 on page 2 - Added “Enabling minPower” on page 9 - Added this Revision History table and the document links on this page

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

